Smart Cane: ShellHacks Project Plan

A navigation and awareness system for blind and low-vision users: an **iPhone 17 Pro mounted on a white cane** (LiDAR + camera) plus an **Insta360 X4 on a hat** (360 awareness). Controlled entirely by voice, with feedback through haptics and spatial audio.

Team: 4 people · **Time:** 36 hours **Target tracks:** Waymo (Map the Future), Google.org (Social Good), Google Cloud (Autonomous AI Agents: ADK/A2A), NVIDIA

---

1. Core Pitch

GPS gets a blind person to the block, not to the door, and it can't see what's in front of them. We built a **Waymo-style autonomy stack for a pedestrian**: perception → prediction → planning. It guides users door-to-door, warns them about hazards before the cane touches them, and makes every trip improve a shared accessibility map for everyone.

One-line differentiator vs. Be My Eyes / Seeing AI: those are on-demand describers; we provide **continuous, proactive guidance**.

---

2. Design Principles (say these in the pitch)

1. **Safety never waits on the network.** Immediate hazards are detected on-device in under 100 ms. The cloud only adds richer awareness.
2. **Don't flood the ears.** Blind users rely on hearing for traffic. Navigation uses directional haptics and spatial audio beacons. Speech is reserved for what matters, and silence is a valid output.
3. **Privacy by design.** Faces are enrolled only with consent, and embeddings are stored encrypted on-device. Conversation notes are **dictated by the user**, never recorded. (Florida requires all-party consent for recording conversations.)
4. **Graceful degradation.** If the connection drops, obstacle warnings keep working and the app announces that detailed descriptions are unavailable.

---

3. Features

MVP (must work in the demo)

- **Voice control:** "Take me to Marston," "Describe what's around me," "Who is this?"
- **Tier 1 hazard detection** (on-device LiDAR + CoreML): drop-offs, poles, people, bikes
- **Turn-by-turn navigation** with directional haptics (Google Routes API)
- **Last-50-feet navigation:** Places and Street View identify the entrance, then vision guides the user to the door

- **360 awareness** via Insta360 → backend: side and rear warnings for approaching cyclists and cars
- **Scene description / peaceful mode** (VLM)
- **Sign and text reading** (room numbers, building names, bus numbers)
- **Consent-based face memory** with user-dictated notes
- **Crowdsourced hazard map:** detected hazards are uploaded and used for route scoring

Stretch goals

- Transit mode: find the stop and read the arriving bus number ("Is this my bus?")
- "Guide me to my ride": read the license plate and walk the user to the rideshare/Waymo door
- Crosswalk assist: detect the walk signal, keep the user aligned, warn about turning cars
- Elevation-aware accessible route scoring (slopes)
- Apple Watch haptics
- Fall detection → SOS with location

---

4. Architecture

Latency tiers

Tier	Where	Latency	Responsibility
Tier 1	iPhone (Neural Engine)	< 100 ms	LiDAR depth + CoreML hazard detection, haptics, TTS, face matching
Tier 2	GPU backend	~150–400 ms round trip	360 detection + tracking (TensorRT), trajectory prediction
Tier 3	GPU backend + model APIs	~0.5–2 s	ADK agent loop, VLM descriptions, sign reading, re-routing

Data flow

- **Production:** Insta360 → phone (Wi-Fi, Insta360 SDK) → cloud → small JSON events → phone → haptics/audio
- **Hackathon:** Insta360 → laptop (USB webcam mode or RTMP) → backend → WebSocket → phone
- **Bandwidth:** never stream raw 360 video. Downscale and send 2–5 fps as compressed JPEGs or low-bitrate H.264.

- **Event format:** `{ "type": "cyclist", "direction": "left-rear", "urgency": "high" }`

Components

Hardware: Insta360 X4 (hat) · laptop bridge · iPhone 17 Pro (cane) · AirPods / Apple Watch (outputs)

iOS app (on-device):

- Voice interface: speech → intents
- Navigation engine: route follower, heading correction, last-50-ft handoff
- Tier 1 safety engine: ARKit LiDAR depth + CoreML detector
- Face recognizer: consent-enrolled only
- Local encrypted store: face embeddings + dictated notes
- Feedback manager: priority queue (safety > navigation > descriptions) → haptics, spatial audio, TTS

GPU backend (FastAPI + ADK):

- 360 stream ingest: equirectangular → multi-view perspective frames
- Perception: detection + tracking (TensorRT)
- Prediction: car and cyclist trajectories
- Scene VLM service
- WebSocket gateway: pushes events to the phone
- Hazard map agent (A2A service)

External: Google Maps Platform · Gemini (Live API) · NVIDIA NIM · Firestore

ADK agent design (Google Cloud track)

- **Guide agent (root `LlmAgent`):** handles voice intents and delegates; user state lives in session state.
- **Awareness loop (`LoopAgent`):** runs for the whole trip. Each iteration:
  1. `ParallelAgent` fans out: Hazard Scout · Traffic Predictor · Sign Reader · Face Matcher
  2. **Synthesizer agent** reads all outputs from shared state (`output_key`), deduplicates, and picks the single most important announcement, or stays silent
  3. Escalates to exit on arrival or cancel
- **Route planner (`LoopAgent`):** re-checks the route against the hazard map + Maps each iteration and re-plans when a hazard appears.

- **Hazard map agent (A2A):** standalone agent with an agent card. The cane reports hazards to it, and other agents query it. Demo it with a second client, e.g. a mock city dashboard or a rideshare agent asking for an accessible pickup point.
- The NVIDIA TensorRT detector is wrapped as a **tool** called by the Hazard Scout, so one component counts for both the NVIDIA and Google Cloud tracks.
- **ADK stays out of the safety-critical path.** It runs at ~1–2 Hz. Tier 1 handles instant hazards.

---

5. Track Talking Points

Waymo: Map the Future

- Framed as a pedestrian autonomy stack (perception → prediction → planning)
- Last-50-feet navigation using Places + Street View
- Crowdsourced hazard map + Elevation API route scoring
- Stretch: "guide me to my Waymo," transit mode

Google.org: Social Good

- Built on Gemini + Google Cloud
- Impact: millions of Americans live with significant vision loss. Guide dogs cost tens of thousands of dollars and have long waitlists. Our hardware is mostly a phone people may already own.
- **Get real user feedback:** contact UF Disability Resource Center. Even one short video of a blind user trying it is huge.
- Responsible AI: consent-based faces, on-device storage, no secret recording

Google Cloud: Autonomous AI Agents

- **Why loop:** the street changes constantly. Only a loop notices new hazards, approaching bikes, and blocked routes, and re-plans.
- **Why parallel:** the perception tasks are independent, so loop latency is set by the slowest task, not the sum. **Benchmark sequential vs. parallel on the same frame and put both numbers on a slide.**
- **How:** `LoopAgent`, `ParallelAgent`, `SequentialAgent`, shared session state via `output_key`, escalation to exit, bidi streaming with Gemini Live, A2A agent card for the hazard map
- Business angle: the hazard map is a data product for cities (ADA compliance, sidewalk repair prioritization)

NVIDIA

- TensorRT-accelerated detection + tracking on the 360 feed

- NVIDIA NIM for the hosted model layer (optional: Riva for speech)
- Show GPU latency and throughput numbers
- *Paste the exact NVIDIA track description and tailor this section.*

---

6. Team Split

Owner	Lane	Track focus
P1: iOS core	Sensor capture, Tier 1 safety engine, feedback manager (haptics, spatial audio, priority queue)	Critical path, so give this to the strongest Swift dev
P2: Backend + GPU	Insta360 ingest, detection/tracking, prediction, WebSocket gateway, **ADK agent backend**, latency benchmarks	NVIDIA + Google Cloud
P3: Maps + navigation	Routes/transit, last-50-ft entrance lookup, **hazard map A2A agent** + database	Waymo
P4: AI experience	Voice intents, VLM + peaceful mode, face memory, pitch deck, demo script	Google.org

---

7. 36-Hour Timeline

Hours	Goal
0–4	Insta360 stream reaching the backend. Bare iOS app ↔ WebSocket. P2 spends early time learning ADK. **If the 360 stream isn't working by hour 4, switch to the phone-only fallback.**
4–16	Everyone builds their lane against mocked inputs. By hour 16: obstacle alerts fire on the phone, a route plays turn-by-turn haptics.
16–24	Integration: backend warnings + VLM → feedback manager, hazards → map. Sleep in shifts.
24–30	Walk the demo route blindfolded, repeatedly. Fix what breaks, cut what's flaky.
30–36	Feature freeze. Record the backup demo video, finish slides (latency numbers, ethics, parallel-vs-sequential benchmark), rehearse the pitch.

---

8. Demo Plan

One flawless route beats ten half-working features.

A blindfolded teammate walks from outside Marston Science Library to a specific building door or bus stop:

1. Voice: "Take me to [destination]"
2. Haptic turn-by-turn guidance begins
3. A planted obstacle triggers a Tier 1 warning
4. A teammate on a bike passes behind them, triggering a 360 rear warning
5. The system reads a sign
6. It recognizes an enrolled teammate: "That's [name], you talked about [dictated note]"
7. The last 50 feet: it guides them to the actual door
8. Show the hazard appearing on the shared map, then an A2A query from a second agent

Demo logistics: run off a phone hotspot (venue Wi-Fi is unreliable), test that exact setup beforehand, and keep a recorded backup video.

---

9. Risks & Fallbacks

Risk	Mitigation
Insta360 live stream won't work	Decide by hour 4; fall back to the iPhone-only demo
Cane sweep makes phone data shaky	Mount high on the handle angled forward + IMU compensation, or use a chest strap
Network latency / drops	Tier 1 is fully on-device; cloud features degrade gracefully
Venue Wi-Fi	Phone hotspot; backend on a cloud VM or a nearby laptop
No GPU available	Sponsor credits → teammate RTX laptop as the "edge server" → hosted APIs only
Face recognition ethics questions	Consent-only enrollment, on-device encrypted storage, dictated notes
Scope creep	MVP list is fixed; stretch goals only after hour 24 if the route demo is solid

---

10. Bill of Materials

Hardware

Item	Qty	Notes
iPhone 17 Pro	1	LiDAR + cameras; primary device
Insta360 X4	1	Borrowed from UF Marston Science Library; bring a charged spare battery if available
microSD card (V30+, 64 GB+)	1	For the Insta360
Cap / hat + camera mount	1	Cap with a 1/4"-20 screw mount or a head-strap mount; secure it so the camera stays level
White cane (folding)	1	Demo prop + phone mount point
Phone clamp mount for cane	1	Bike handlebar-style clamp fits a cane shaft; must hold the phone rigidly
Chest-strap phone mount	1	Backup if the cane mount is too shaky
AirPods Pro (or AirPods with head tracking)	1	Spatial audio beacons + TTS
Apple Watch	0–1	Optional, for wrist haptics
Mac with Xcode	1+	**Required** to build and install the iOS app
Laptop with NVIDIA RTX GPU	0–1	Local "edge server" fallback / Insta360 bridge
USB-C cables (C-to-C, C-to-A)	3+	Insta360 webcam mode, phone charging
Power banks	2	Phone + Insta360 drain fast when streaming
Phone with hotspot plan	1	Demo connectivity
Blindfold	1	Demo
Obstacle props (cone, box, chair)	2–3	Planted hazards for the demo

Accounts & API keys (set up before the hackathon)

- **Google Cloud project** with billing or sponsor credits

- **Google Maps Platform** key with Routes API, Places API (New), Street View Static API, Elevation API, and Geocoding API enabled
- **Gemini API key** (Google AI Studio or Vertex AI)
- **NVIDIA API key** (build.nvidia.com, for NIM)
- **Firebase project** (Firestore) for the hazard map
- **Apple ID** for signing the app onto the iPhone (a free account works for device testing; builds expire after 7 days)
- **GitHub repo** + a shared Figma board for the diagram

iOS (Swift / SwiftUI)

Framework / library	Use
SwiftUI	App UI (minimal; mostly for judges watching the screen)
ARKit (`sceneDepth`)	LiDAR depth maps
CoreML	On-device object detection
Ultralytics YOLO (exported to CoreML)	Detector model for Tier 1
Vision	Face detection, text recognition (`VNRecognizeTextRequest`)
MobileFaceNet / FaceNet (CoreML)	Face embeddings. Vision detects faces but doesn't provide identity embeddings, so a separate model is needed
Speech (`SFSpeechRecognizer`)	Voice commands
AVFoundation (`AVSpeechSynthesizer`, `AVAudioEngine`, `AVAudioEnvironmentNode`)	TTS + spatial audio beacons
Core Haptics	Directional haptic patterns
CoreLocation + CoreMotion	GPS, heading, IMU (cane sweep compensation)
`URLSessionWebSocketTask`	Backend event stream
CryptoKit + Keychain / SwiftData	Encrypted local face + notes store
WatchConnectivity	Optional Apple Watch haptics
Google Maps SDK for iOS	Optional map view for the demo screen

Backend (Python 3.11+)

Library / tool	Use
FastAPI + Uvicorn	API + WebSocket gateway
`google-adk`	Agent Development Kit (Loop/Parallel/Sequential agents)
`a2a-sdk`	A2A protocol server for the hazard map agent
`google-genai`	Gemini API client (incl. Live API)
MediaMTX	RTMP server to receive the Insta360 stream
FFmpeg	Stream decoding / frame extraction
OpenCV (`opencv-python`)	Frame processing
`py360convert`	Equirectangular → perspective views
Ultralytics YOLO + ByteTrack	Detection + multi-object tracking
PyTorch + TensorRT	GPU inference (export YOLO to a TensorRT engine)
NumPy	Trajectory math / prediction
`firebase-admin`	Firestore writes for the hazard map
`googlemaps` or direct REST calls	Server-side Maps API calls
`httpx`	Calls to NVIDIA NIM endpoints

Infrastructure

Item	Use
GCP Compute Engine GPU VM (L4 / T4) or Cloud Run with GPU	Hosts the backend
Firestore	Hazard map database
ngrok or Cloudflare Tunnel	Expose a laptop backend if not using a cloud VM
Docker	Consistent backend environment

Pitch & demo

- Figma (architecture diagram)
- Slide deck with the latency-tier table, the parallel-vs-sequential benchmark, and ethics/privacy points
- Screen recording (QuickTime / iPhone screen recording) for the backup demo video

---

11. Pre-Hackathon Checklist

- [ ] Pick up the Insta360 X4; update firmware; **test USB webcam mode and RTMP streaming to a laptop**
- [ ] Enable all Google Cloud / Maps APIs and confirm the keys work
- [ ] Check whether sponsors are offering GCP or NVIDIA credits
- [ ] Confirm someone has a Mac with Xcode and can deploy to the iPhone 17 Pro
- [ ] Export a YOLO model to CoreML and test it on the phone
- [ ] P2: go through the ADK quickstart + one LoopAgent/ParallelAgent example
- [ ] Buy or borrow the cane, mounts, hat mount, and power banks
- [ ] Walk and pick the demo route near Marston
- [ ] Reach out to UF Disability Resource Center for user feedback
- [ ] Get the exact NVIDIA track description and tailor that section
- [ ] Verify current Gemini model names and ADK/A2A API signatures in the docs